

Proyecto 1 (%)

Fecha de Entrega:

Servidor HTTP

1 Objetivo del Proyecto

El presente proyecto tiene como objetivo que los estudiantes demuestren su comprensión profunda de los fundamentos de los sistemas operativos mediante la implementación de un servidor HTTP basado en la especificación HTTP/1.0, sin capa de cifrado (HTTP puro), que pueda atender múltiples clientes simultáneamente. A través de esta experiencia, el estudiantado desarrollará competencias técnicas relacionadas con el manejo de procesos, concurrencia, sincronización, planificación, colas de trabajo y uso de sockets en sistemas Unix-like.

Se espera que el desarrollo del proyecto integre prácticas profesionales de ingeniería, incluyendo pruebas unitarias con un *coverage* mínimo del 90%, documentación técnica clara, diseño modular y elaboración de un informe científico completo que detalle los experimentos de desempeño bajo carga. Este proyecto fomenta la aplicación práctica de conocimientos teóricos, incentivando la reflexión crítica, el trabajo autónomo y la producción técnica de alta calidad.

2 Descripción del Proyecto

En el contexto de **RedUnix S.A.**, se requiere un servidor HTTP/1.0 funcional (sin HTTPS) capaz de recibir múltiples conexiones concurrentes de herramientas como *curl* o *Postman*. Cada solicitud debe ser interpretada y redirigida a un *worker* especializado según el tipo de comando solicitado. Para permitir escalabilidad horizontal, se podrá instanciar una cantidad arbitraria (n) de procesos/hilos/goroutines por tipo de comando.

El servidor debe implementar, como mínimo, los siguientes comandos (existentes) accesibles por rutas HTTP:

- `/fibonacci?num=N`
- `/createfile?name=filename&content=text&repeat=x`
- `/deletefile?name=filename`
- `/status` (ver formato en Req. Funcionales)
- `/reverse?text=abcdef`
- `/toupper?text=abcd`

- /random?count=n&min=a&max=b
- /timestamp
- /hash?text=someinput
- /simulate?seconds=s&task=name
- /sleep?seconds=s
- /loadtest?tasks=n&sleep=x
- /help

Endpoints de procesamiento intensivo

Para evaluar **conurrencia, planificación, colas y robustez**, deberán agregarse los siguientes mínimos (CPU-bound e IO-bound). **Nota:** Ninguno puede implementarse *sólo* con `sleep()`; deben realizar trabajo real (cálculo o E/S).

CPU-bound

- /isprime?n=NUM: Primalidad por división hasta $\sqrt{n}$ o prueba de Miller–Rabin (configurable).
- /factor?n=NUM: Factorización en primos; retornar arreglo con factores y conteos.
- /pi?digits=D: Cálculo de π con *Spigot* o Chudnovsky (versión iterativa, control de tiempo).
- /mandelbrot?width=W&height=H&max_iter=I: Genera **mapa de iteraciones** (matriz entera) en JSON. (Opcional: volcar PGM/PPM a disco con nombre.)
- /matrixmul?size=N&seed=S: Multiplica dos matrices $N \times N$ pseudoaleatorias; retorna *hash* SHA-256 del resultado para verificación.

IO-bound

- /sortfile?name=FILE&algo=merge|quick: Ordena en archivo (números enteros, uno por línea). Debe manejar archivos ≥ 50 MB. Retorna 200 + métricas de tiempo.
- /wordcount?name=FILE: Cuenta líneas, palabras y bytes (tipo wc); soporta archivos grandes.
- /grep?name=FILE&pattern=REGEX: Devuelve número de coincidencias y primeras 10 líneas que coinciden.
- /compress?name=FILE&codec=gzip|xz: Comprime el archivo a .gz o .xz; retorna nombre de salida y tamaño.
- /hashfile?name=FILE&algo=sha256: Calcula el hash del archivo; retorna hex.

Métricas y tuning

- /metrics: JSON con tiempos promedio (espera/ejecución), desviación estándar, tamaño de colas por comando, y número de workers activos/ocupados por tipo.

Modelo de trabajos (Jobs) para tareas largas

Para tareas que puedan exceder el **timeout interactivo** (p.ej., 5–15 s) de HTTP/1.0, se deberá implementar un *Job Manager* con colas internas, identificadores y *polling*. Todas las rutas CPU/IO soportarán ejecución directa (*best effort*) y vía job.

Endpoints de Jobs

- `/jobs/submit?task=TASK&<params>`: Encola un trabajo y devuelve `{"job_id": "...", "status": "queued"}`.
- `/jobs/status?id=JOBID`: Devuelve

```
{
  "status": "queued|running|done|error|canceled",
  "progress": 0..100,
  "eta_ms": ...
}
```

- `/jobs/result?id=JOBID`: Devuelve el JSON propio del comando si done; si error, incluir `{"error": "..."}.`
- `/jobs/cancel?id=JOBID`: Intenta cancelar; devuelve estado canceled o not_cancelable.

Semántica mínima requerida

- **Planificación**: FIFO por defecto. Debe permitirse **prioridad** (`prio=low|normal|high`) y **límite de concurrencia** por tipo de comando.
- **Backpressure**: Si cola supera umbral configurable, devolver 503 Service Unavailable con `{"retry_after_ms": ...}`.
- **Timeouts**: Tiempo máx. por trabajo configurable (p.ej. 60s CPU, 120s IO). Al excederse: `error=timeout`.
- **Persistencia efímera**: Los metadatos de jobs deben sobrevivir al menos a un *graceful restart* (archivo temporal o *journal simple*).

3 Requerimientos Funcionales

1. **Múltiples clientes concurrentes** sin bloquear el hilo principal.
2. **Compatibilidad HTTP/1.0** con GET. (Opcional: HEAD.)
3. **Manejo de rutas** (todas las originales + las de procesamiento intensivo).
4. **Escalabilidad por comando**: *Múltiples workers* por tipo, configurables por CLI/env.

5. **/status** en JSON con: *uptime*, PID, conexiones atendidas, *workers* por comando (PID/estado), tamaño de colas.
6. **Modelo de Jobs** para tareas largas: `/jobs/submit|status|result|cancel` con semántica descripta.
7. **Códigos HTTP adecuados**: 200, 400, 404, 409, 429, 500, 503. Incluir mensaje JSON claro.
8. **Parámetros inválidos**: Validación estricta y mensajes de error útiles.
9. **Trazabilidad**: Incluir X-Request-Id y X-Worker-Pid en respuestas.
10. **Configuración**: Puerto, número de workers por comando, *queue depth*, *timeouts*, ruta de data/ por CLI (`-port`, `-workers.isprime=4`, etc.) o variables de entorno.

4 Requerimientos Técnicos

Lenguajes permitidos

- **C**: uso explícito de `fork()`, `socket()`, `bind()`, `listen()`, `accept()` y POSIX.
- **Rust**: `std::net`, `std::thread/async`, canales `mpsc` y `Arc/Mutex`.
- **Go**: `net` y `goroutines`; no usar `net/http`.

Concurrencia y sincronización

- Diseñar **pooles por comando** y colas thread-safe.
- Evitar `sleep()` para sincronización; usar primitivas (`mutex`, `condvars`, canales).
- Recursos compartidos (contadores, archivos) deben ser **seguros y consistentes**.

Restricciones

- Prohibido usar servidores HTTP embebidos o librerías de alto nivel.
- Código desde cero, modular, con separación de responsabilidades.
- Cobertura $\geq 90\%$ con herramientas del lenguaje (`gcov`, `tarpaulin`, `go test -cover`).
- `README.md` con compilación, ejecución, pruebas y arquitectura.

5 Pruebas Unitarias, Integración y Desempeño

- Casos exitosos y fallidos por cada ruta (parámetros límite, malformados).
- **Pruebas de carrera**: Conexiones simultáneas (`N_clients`), verificar que no hay *deadlocks* ni *data races*.
- **Pruebas de colas**: Encolar $\geq 2N$ trabajos donde *N* es el número de workers por comando; verificar `/jobs/status` y `/metrics`.

- **Pruebas de IO grande:** /sortfile, /compress, /hashfile con archivos ≥ 50 MB.
- **Pruebas de CPU:** /pi, /matrixmul, /mandelbrot con tamaños que tomen varios segundos.
- **Métricas:** Reportar latencias p50/p95/p99 y throughput para 3 perfiles de carga.
- Un solo comando para ejecutar todas las pruebas; salida interpretable.

6 Especificación de Respuestas JSON (extracto)

- /isprime?n=97 →
`{"n":97,"is_prime":true,"method":"miller-rabin","elapsed_ms":12}`
- /factor?n=360 →
`{"n":360,"factors":[[2,3],[3,2],[5,1]],"elapsed_ms":7}`
- /matrixmul?size=800 →
`{"size":800,"seed":123,"result_sha256":"...", "elapsed_ms":...}`
- /sortfile?name=data.txt&algo=merge →
`{"file":"data.txt","algo":"merge","sorted_file":"data.sorted","elapsed_ms":...}`
- /jobs/submit?task=isprime&n=... →
`{"job_id":"...", "status":"queued"}`
- /jobs/status?id=... →
`{"status":"running","progress":42,"eta_ms":3800}`
- /metrics →
`{"queues":{"isprime":3,...},"workers":{"isprime":{"total":4,"busy":2}}, "latency_ms":{"...`

7 Informe Científico

- Resumen, Introducción, Marco Teórico, Diseño e Implementación, Estrategia de Pruebas, Resultados (p50/p95/p99, colas, *scalability*), Discusión, Conclusiones, Referencias.
- Comparar **CPU-bound vs IO-bound**, efecto de tamaño de pool, profundidad de cola y política de planificación.

Entregables

1. **Código fuente** (Rust, C o Go).
2. **Documentación** técnica y **Manual de usuario**.
3. **Pruebas** unitarias/integración con cobertura $\geq 90\%$ y reporte.
4. **Evidencias** (capturas/logs de curl/Postman) incluyendo rutas de jobs y métricas.
5. **Datasets y scripts** para generar archivos grandes (data/).

8 Rúbrica de Evaluación

Criterio	Descripción	Puntaje Máx.
Funcionamiento General	Responde correctamente a todas las rutas (básicas + intensivas).	10
Concurrencia y Sincronización	Pooles por comando, ausencia de <i>data races/deadlocks</i> .	10
Modelo de Jobs	/jobs/* completo: encolado, estado, resultado, cancelación, timeouts.	10
Planificación y Backpressure	Prioridades, límites de concurrencia, 429/503 con <code>retry_after</code> .	10
Procesamiento CPU/IO Real	Tareas no triviales (sin <code>sleep()</code>), manejo de archivos grandes y cómputo intensivo.	10
Pruebas & Cobertura	$\geq 90\%$, pruebas de carga, colas y estrés.	10
Métricas y Observabilidad	/status, /metrics, trazas, X-Request-Id.	10
Diseño Modular y Docs	Arquitectura clara, separaciones de responsabilidad, README completo.	10
Informe Científico	Análisis de latencias p50/p95/p99, escalabilidad y discusión crítica.	10
Estilo Profesional	Buenas prácticas de código y versionado.	10
Total		100

Cuadro 1
Rúbrica de Evaluación del Proyecto

Recomendaciones

- **CLIs reproducibles:** parámetros como `-workers.isprime=4 -queue.isprime=64 -timeout.cpu=60000`.
- **Datos sintéticos:** generar archivos grandes de forma determinística (semillas).
- **Perfilamiento:** medir CPU (`time`), IO (`iostat`), y memoria del proceso.